Welcome to your first Session in Level 2

TypeScript in a nutshell

1 0 1 1 0 1 1 0 1 1 0 1 1 0 0 1 1 0 1 1 0 1 1 0 1 1 0 1 1 1 0 1 1 0 1 1 0 1 1 1 1 1 0 1

Objectives

By the end of this session, You will be able to:

1. Understand why TypeScript exists and how it evolved.
2. Learn the difference between static vs dynamic typing.
3. Explore TypeScript's main features (types, inference, interfaces, generics).
4. Learn what transpilers do and how TSC works.
5. Configure TypeScript using tsconfig.json.
6. See how TypeScript fits into modern build workflows.

Don't forget our Rules

1. 15 mins late is acceptable, after 15 mins every minute is equal to 1 pound. max(20 pounds)

2. Absence without any excuse -> 2 warnings

3. Missing Task without any excuse -> 1 warning

4. Any unacceptable behavior -> warning

Don't forget our Rules

1. Any correct answer in the session = Mohsens
2. Best member of the week will be awarded next session
3. Best of the month will be awarded 1/12 based on mohsens
4. Best of the Season will be awarded at the end of the season based on mohsens and overall performance

01 Recap

- What happened last Session?
- What did we talk about?
- What was the benefit you got from last session?

1 0 1 1 0 1 1 0 1 1 0 1 1 0 0 1 1 0 1 1 0 1 1 0 1 1 0 1 1 1 0 1 1 0 1 1 0 1 1 1 1 1 0 1

Last Session Table of Contents

01

Student activities

What are they?
What they do?

02

What is OSC?

Who are we?
What is Open Source?
History of OSC

03

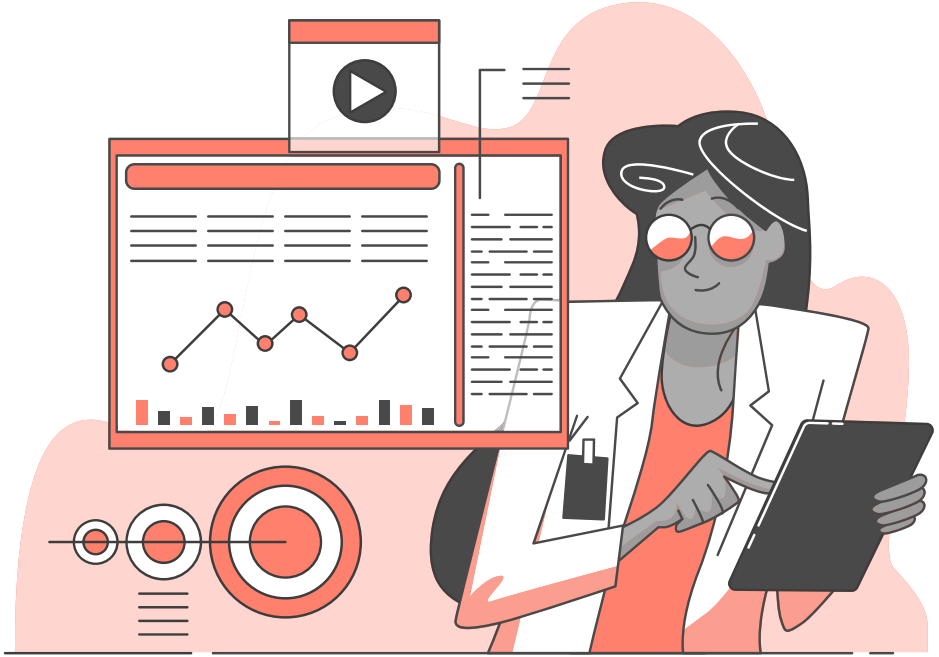
OSC Hierarchy

Who is the Board?
Who is the High Board?

04

Backend Committee

History
What do we do?
Committee Structure
Structure of each Session
Outline of each Level & Phases



Last Session Table of Contents

05

Backend Committee

Punishment System
Rewarding System
Reporting System

06

Backend

What is Backend?
Why Backend?
How to be Backend Engineer?

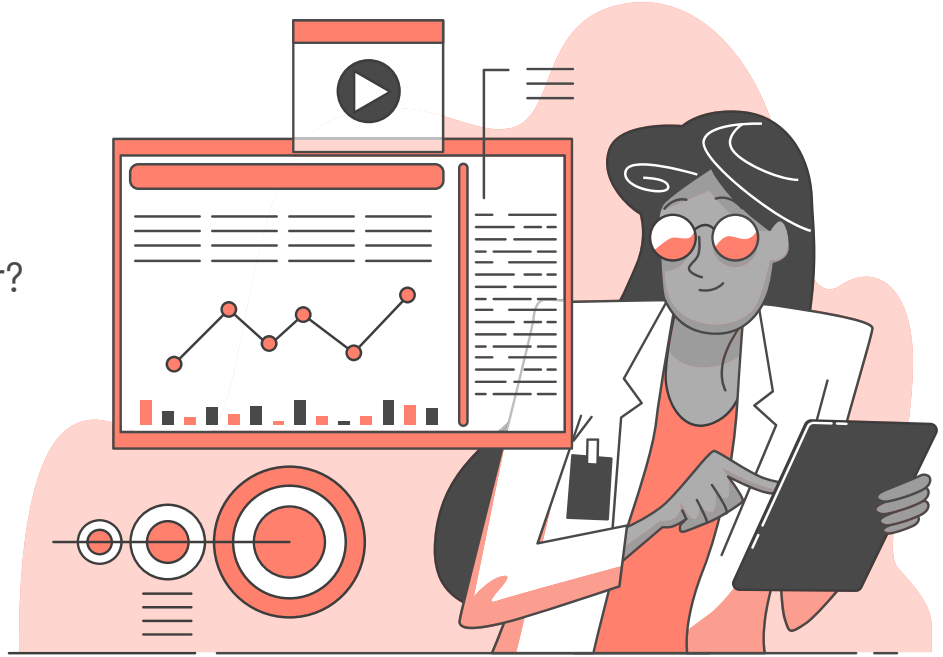
07

Why we are here?

The value of what we do
How to get the most benefit?
What to expect from us?

08

Party!!!!!

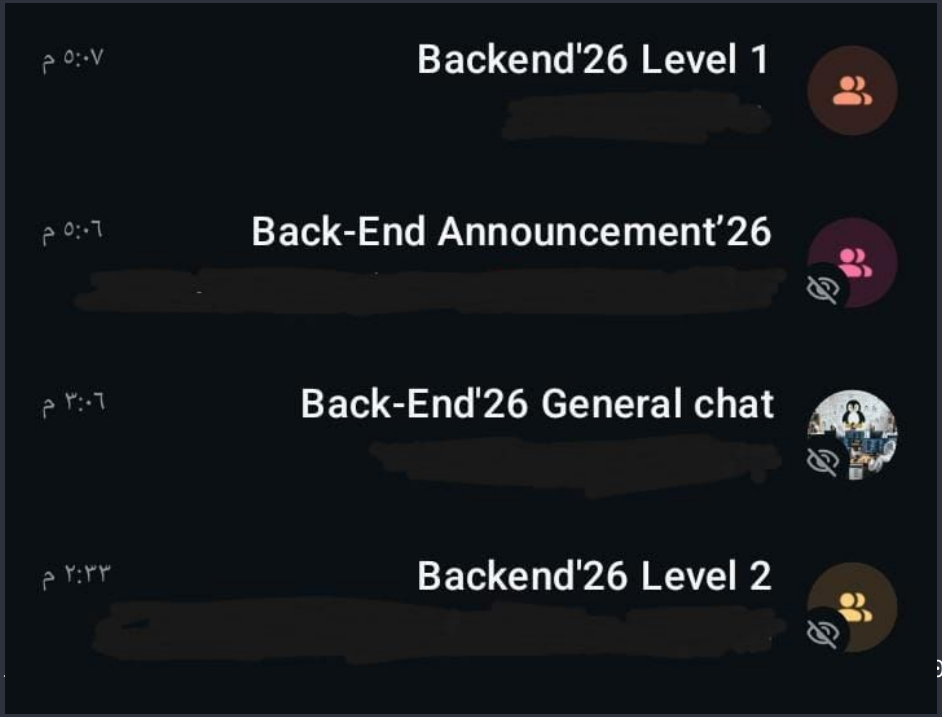


02 Tour

- Whatsapp Groups
- Github
- Discord
- Google Sheet
- Forms

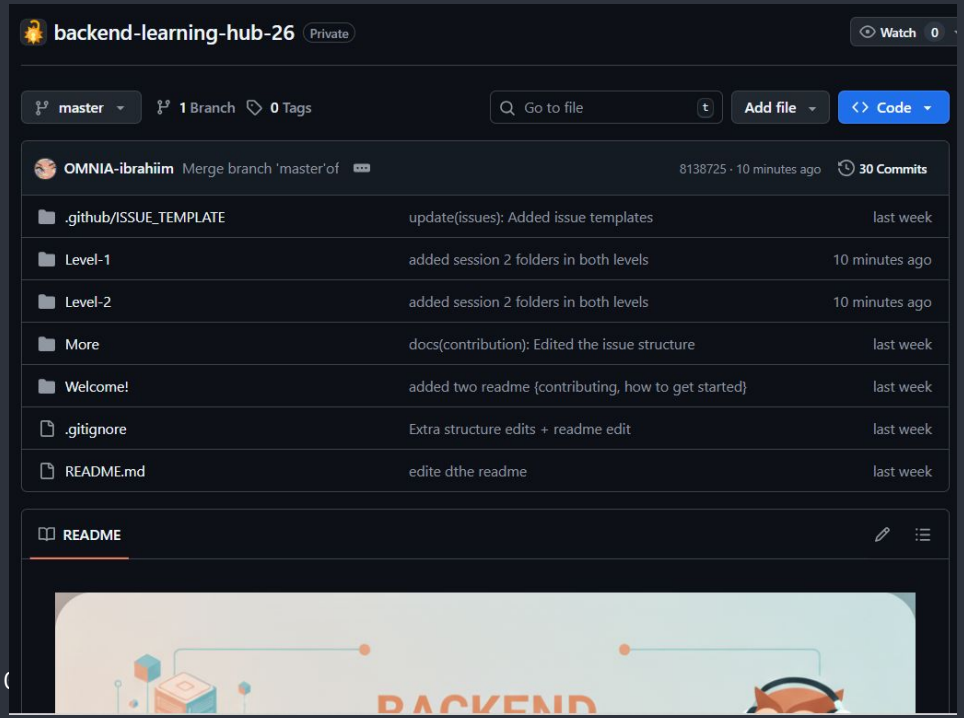
1 0 1 1 0 1 1 0 1 1 0 1 1 0 0 1 1 0 1 1 0 1 1 0 1 1 0 1 1 1 0 1 1 0 1 1 0 1 1 1 1 1 0 1

02 Tour: Whatsapp Groups



1 0 1 1 0 1 1 0 1 1 1 0 1 1 1 0 1 1 1 0 1

Tour: Github



1 0 1 1 0 1 1 0 1 1 0 1 1 1 0 1 1 1 1 1 0 1

02

Tour: Discord

BACK-END

- badgician 🏆
- backend-announcements
- backend-materials
- backend-tasks
- backend-chat
- backend-ideas
- backend-opportunities NEW
- backend-news NEW
- backend-memories NEW
- Meeting Room 1
- Meeting Room 2
- Board Room

1 0 1 1 0 1 1 0 1 1 0 1 1 0 0 1 1 0 1 0 1 1 1 1 1 0 1

02

Tour: Sheet

A	B	C	D
	Important Links		
	Link	Description	
	https://github.com/Open-Source-Community	Open source community in github	
	https://github.com/Open-Source-Community/backend-learning-hub-26	Backend learning hub repo (on osc)	
	https://forms.gle/RRSP8fFHmbqVwzhW9	Feedback Form	
		ideas Form	
	https://chat.whatsapp.com/GOtOUVA9DjmKbkxFE8IrTR	Backend Genral chat Whatsapp Group	
	https://chat.whatsapp.com/IA3AuYf79JO36haYQBiWOt	Backend Announcment chat Whatsapp Group	
	https://www.facebook.com/groups/osccommunitytalk	OSC Community talk Facebook	
	https://www.facebook.com/groups/osccrew	OSC Crew Facebook	
	https://discord.gg/bBjmuBpn	Discord Channel	
	https://drive.google.com/drive/folders/1z8mHX6u	Stickers Drive	

1 0 1 1 0 1 1 0 1 1 1 1 1 0 1

Board

02

Tour: Forms

- *Feedback Form*
- *Excuses Form*
- *Attendance Form*

1 0 1 1 0 1 1 0 1 1 0 1 1 0 0 1 1 0 1 1 0 1 1 0 1 1 0 1 1 1 0 1 1 0 1 1 0 1 1 1 1 1 0 1

Table of Content

1_ Problem with JS

2_ How the Problem is Solved

3_ What is TypeScript

4_ Basics of TypeScript

- ★ Types
- ★ Functions
- ★ Interfaces
- ★ Union Types
- ★ Intersection Types
- ★ Generic Types
- ★ Any
- ★ Type vs Interface vs Class vs Object

Table of Content

5_ code Structure Different Philosophies

6_ How TS is Transpiled

7_ How to set it up

8_ What Files Matters to us

Who here hates JS?

1 0 1 1 0 1 1 0 1 1 0 1 1 0 0 1 1 0 1 1 0 1 1 0 1 1 0 1 1 1 0 1 1 0 1 1 0 1 1 1 1 1 0 1

</ JS development:

- JavaScript is awesome for building things, but it can get chaotic.
- Suddenly you're debugging for hours, and the bug?
- Something JS didn't warn you about until it exploded at runtime.

1 0 1 1 0 1 1 0 1 1 0 1 1 0 0 1 1 0 1 1 0 1 1 0 1 1 0 1 1 1 0 1 1 0 1 1 0 1 1 1 1 1 0 1

How was this problem fixed?

1 0 1 1 0 1 1 0 1 1 0 1 1 0 0 1 1 0 1 1 0 1 1 0 1 1 0 1 1 1 0 1 1 0 1 1 0 1 1 1 1 1 0 1

</ TypeScript was the answer

Microsoft introduced TypeScript as a solution:

- Code is safe and type annotated
- Bugs appear early in compile time
- Autocomplete is provided
- Maintaining Large projects became easier.

What is TypeScript?

1 0 1 1 0 1 1 0 1 1 0 1 1 0 0 1 1 0 1 1 0 1 1 0 1 1 0 1 1 1 0 1 1 0 1 1 0 1 1 1 1 1 0 1

</ TypeScript, JavaScript's cousin

TypeScript is a superset of JavaScript, Where it includes static types, and is strongly typed, and can also be loosely typed.

TypeScript is also not "Compiled" but rather Transpiled – converted into JavaScript code, but why?

</ Quick fact

- When TypeScript was created, no browser or runtime could execute TypeScript directly.

- So TypeScript had two choices:
  - Option A:
    - Create a new runtime or virtual machine that understands TypeScript → Not practical.
    - It would break compatibility with the entire JS ecosystem.
  - Option B:
    - Compile TypeScript into JavaScript, which browsers already understand.
    - TypeScript chose 100% compatibility with existing JavaScript engines, so it had to be transpiled.

</ Dynamic vs Static Typing

- Dynamic typing means type checking happens at runtime, not during compilation. (e.g. Python, JavaScript)

- While Static typing is that the type checking happens during writing code, so errors appear early on. (e.g. C++, Java)

Quick quest

Why do we say compilation time even tho JS is interpreted and what is a LSP?

1 0 1 1 0 1 1 0 1 1 0 1 1 0 0 1 1 0 1 1 0 1 1 0 1 1 0 1 1 1 0 1 1 0 1 1 0 1 1 1 1 1 0 1

Table of Content

~~1_ Problem with Js~~

~~2_ How the Problem is Solved~~

~~3_ What is TypeScript~~

4_ Basics of TypeScript

- ★ Types
- ★ Functions
- ★ Interfaces
- ★ Union Types
- ★ Intersection Types
- ★ Generic Types
- ★ Any
- ★ Type vs Interface vs Class vs Object

1 0 1 1 0 1 1 0 1 1 0 1 1 0 0 1 1 0 1 1 0 1 1 0 1 1 0 1 1 1 0 1 1 0 1 1 0 1 1 1 1 1 0 1

TypeScript Basics

1 0 1 1 0 1 1 0 1 1 0 1 1 0 0 1 1 0 1 1 0 1 1 0 1 1 0 1 1 1 0 1 1 0 1 1 0 1 1 1 1 1 0 1

1- Types:

```
let Name: string = "Mohsen";
let Age: Number = 28;
let isCool: Boolean = true;
```

You can also define your own type, for example:

```
type Committee = "Backend" | "Linux" | "Frontend";
```

1 0 1 1 0 1 1 0 1 1 0 1 1 0 0 1 1 0 1 1 0 1 1 0 1 1 0 1 1 1 0 1 1 0 1 1 0 1 1 1 1 1 0 1

2- Functions:

```
function functionName(Parameters: parameterType): ReturnType {
    Function body
}
```

Example:

```
function Greet(name: string): void {
  console.log(`Hello ${name}`)
}
```

1 0 1 1 0 1 1 0 1 1 0 1 1 0 0 1 1 0 1 1 0 1 1 0 1 1 0 1 1 1 0 1 1 0 1 1 0 1 1 1 1 1 0 1

3- Interfaces:

Think of them as Structs in C++

```
interface member {
   name: String;
   age: Number;
   committee: String;
}

const member = { name:"Mohsen", age:28, committee:"Linux" }
```

1 0 1 1 0 1 1 0 1 1 0 1 1 0 0 1 1 0 1 1 0 1 1 0 1 1 0 1 1 1 0 1 1 0 1 1 0 1 1 1 1 1 0 1

4- Union Types:

```
let age: Number | String = "8";
```

As you can see we can here use the pipe operator “ | ” to declare that a variable may have 2 different data types.

1 0 1 1 0 1 1 0 1 1 0 1 1 0 0 1 1 0 1 1 0 1 1 0 1 1 0 1 1 1 0 1 1 0 1 1 0 1 1 1 1 1 0 1

Table of Content

~~1_ Problem with Js~~

~~2_ How the Problem is Solved~~

~~3_ What is TypeScript~~

4_ Basics of TypeScript

- ~~Types~~
- ~~Functions~~
- ~~Interfaces~~
- ~~Union Types~~
- Intersection Types
- Generic Types
- Any
- Type vs Interface vs Class vs Object

1 0 1 1 0 1 1 0 1 1 0 1 1 0 0 1 1 0 1 1 0 1 1 0 1 1 0 1 1 1 0 1 1 0 1 1 0 1 1 1 1 1 0 1

5- Intersection Types:

```
type Id = { id: number };
type Name = { name: string };

type User = Id & Name;
```

Here use the ampersand operator " & " to declare that a variable will have the attributes of all types declared.

1 0 1 1 0 1 1 0 1 1 0 1 1 0 0 1 1 0 1 1 0 1 1 0 1 1 0 1 1 1 0 1 1 0 1 1 0 1 1 1 1 1 0 1

6- Generic Types:

```
function identity<T>(value: T): T {
  return value;
}

let num = identity<number>(42);     // T is number
let str = identity<string>("hello"); // T is string
let auto = identity(true);          // T inferred as boolean
```

This is like generalizing what a function can do to multiple data types.

1 0 1 1 0 1 1 0 1 1 0 1 1 0 0 1 1 0 1 1 0 1 1 0 1 1 0 1 1 1 0 1 1 0 1 1 0 1 1 1 1 1 0 1

extra- WD-40 of TypeScript:

The Any keyword is used when you don't expect a certain data type, like what JavaScript does.

Example:

```
let age: any;
age = "hello world";
age = 32;
```

1 0 1 1 0 1 1 0 1 1 0 1 1 0 0 1 1 0 1 1 0 1 1 0 1 1 0 1 1 1 0 1 1 0 1 1 0 1 1 1 1 1 0 1

TypeScript: Type vs Interface vs Class vs Object

Type	Interface	Class	Object
Compile-Time Only	Compile-Time Only	Run time	Run time
Disappears after compilation as it is unique for TS	Disappears after compilation as it is unique for TS	Exists at runtime, in the JS code	Exists at runtime, in the JS code
A way to define shapes and aliases for type checking	A contract that defines object structure	A blueprint for creating objects with behavior	Actual data stored in memory

1 0 1 1 0 1 1 0 1 1 0 1 1 0 0 1 1 0 1 1 0 1 1 0 1 1 0 1 1 1 0 1 1 0 1 1 0 1 1 1 1 1 0 1

Another Quest

What is partial type in TypeScript?

1 0 1 1 0 1 1 0 1 1 0 1 1 0 0 1 1 0 1 1 0 1 1 0 1 1 0 1 1 1 0 1 1 0 1 1 0 1 1 1 1 1 0 1

```javascript
function CalculateArea [Length, Width] {
   return Length*Width;
}

let area = CalculateArea[2,3]; // Works fine
area = CalculateArea[2,"3"];   // Works fine (isn't supposed to)
```

JavaScript

```typescript
function CalculateArea [Length: number, Width: number]: number {
   return Length*Width;
}

let area = CalculateArea[2,3];   // Works fine
area = CalculateArea[2,"3"];     // Compile Time error
```

TypeScript

1 0 1 1 0 1 1 0 1 1 0 1 1 0 0 1 1 0 1 1 0 1 1 0 1 1 0 1 1 1 0 1 1 0 1 1 0 1 1 1 1 1 0 1

Table of Content

~~1_ Problem with Js~~

~~2_ How the Problem is Solved~~

~~3_ What is TypeScript~~

~~4_ Basics of TypeScript~~

- ~~Types~~
- ~~Functions~~
- ~~Interfaces~~
- ~~Union Types~~
- ~~Intersection Types~~
- ~~Generic Types~~
- ~~Any~~
- ~~Type vs Interface vs Class vs Object~~

1 0 1 1 0 1 1 0 1 1 0 1 1 0 0 1 1 0 1 1 0 1 1 0 1 1 0 1 1 1 0 1 1 0 1 1 0 1 1 1 1 1 0 1

Table of Content

5_ code Structure Different Philosophies

6_ How TS is Transpiled

7_ How to set it up

8_ What Files Matters to us

1 0 1 1 0 1 1 0 1 1 0 1 1 0 0 1 1 0 1 1 0 1 1 0 1 1 0 1 1 1 0 1 1 0 1 1 0 1 1 1 1 1 0 1

</ Top-down vs Bottom-up flows

Top-Down

```
interface User {
  id: number;
  name: string;
  address: Address;
  roles: Role[];
}

interface Address {
  city: string;
  street: string;
}
type Role = "admin" | "member" | "guest";
```

Bottom-up

```
interface Id {
  id: number;
}
interface Name {
  name: string;
}
type Role = "admin" | "member" | "guest";

interface User extends Id, Named {
  roles: Role[];
}
```

Bottom up	Top down
Start small then build up	Start big then break down
Build all pieces then assemble	Write what you want first then define how later

1 0 1 1 0 1 1 0 1 1 0 1 1 0 0 1 1 0 1 1 0 1 1 0 1 1 0 1 1 1 0 1 1 0 1 1 0 1 1 1 1 1 0 1

</ Shape Compatibility

If we define multiple interfaces that have common attributes we can assign an object of the more general interface to an object of the narrower scope interface.

1 0 1 1 0 1 1 0 1 1 0 1 1 0 0 1 1 0 1 1 0 1 1 0 1 1 0 1 1 1 0 1 1 0 1 1 0 1 1 1 1 1 0 1

Shape Compatibility example

```
interface PowerSocket {
  voltage: number;
  frequency: number;
}

function plugIn(socket: PowerSocket) {
  console.log(`Connected to ${socket.voltage}V
socket`);
}

const euSocket = {
  voltage: 230,
  frequency: 50,
  country: "Germany",
  grounded: true
}
```

```
const usSocket = {
  voltage: 120,
  frequency: 60,
  outlets: 2
};

const egyptSocket = {
  voltage: 220,
  frequency: 50,
  type: "C/F"
};

plugIn(euSocket);
plugIn(usSocket);
plugIn(egyptSocket);
```

Table of Content

~~5_ code Structure Different Philosophies~~

6_ How TS is Transpiled

7_ How to set it up

8_ What Files Matters to us

1 0 1 1 0 1 1 0 1 1 0 1 1 0 0 1 1 0 1 1 0 1 1 0 1 1 0 1 1 1 0 1 1 0 1 1 0 1 1 1 1 1 0 1

Hands On

1 0 1 1 0 1 1 0 1 1 0 1 1 0 0 1 1 0 1 1 0 1 1 0 1 1 0 1 1 1 0 1 1 0 1 1 0 1 1 1 1 1 0 1

Break

1 0 1 1 0 1 1 0 1 1 0 1 1 0 0 1 1 0 1 1 0 1 1 0 1 1 0 1 1 1 0 1 1 0 1 1 0 1 1 1 1 1 0 1